File operations- handling text and binary files – read - write - seek, command-line arguments.

File

- A file is a place on the disk where a group of related data is stored.
- naming a file, opening a file, reading data from a file, writing data to a file, and closing a file.
- **Text file** – A text file contains data in the form of ASCII characters and is generally used to store a stream of characters. Each line in a text file ends with a new line character ("\n"), and generally has a ".txt" extension.
- **Binary file** – A binary file contains data in raw bits (0 and 1). Different application programs have different ways to represent bits and bytes and use different file formats. The image files (.png, .jpg), the executable files (.exe, .com), etc. are the examples of binary files.

Function name	*Operation*
fopen()	* Creates a new file for use.
	* Opens an existing file for use.
fclose()	* Closes a file which has been opened for use.
getc()	* Reads a character from a file.
putc()	* Writes a character to a file.
fprintf()	* Writes a set of data values to a file.
fscanf()	* Reads a set of data values from a file.
getw()	* Reads an integer from a file.
putw()	* Writes an integer to a file.
fseek()	* Sets the position to a desired point in the file.
ftell()	* Gives the current position in the file (in terms of bytes from the start).
rewind()	* Sets the position to the beginning of the file.

FILE Pointer

- While working with **file handling**, you need a **file pointer** to store the reference of the **FILE** structure returned by the fopen() function. The file pointer is required for all file-handling operations.
- The fopen() function returns a pointer of the **FILE** type. **FILE** is a predefined struct type in stdio.h and contains attributes such as the file descriptor, size, and position, etc.

Opening (Creating) a File

```
FILE *fp = fopen("filename", "mode");
```

Text File Modes

Mode	
"r"	Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer that points to the first character in it. If the file cannot be opened fopen( ) returns NULL.
"w"	Open for writing in text mode. If the file exists, its contents are overwritten. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file.
"a"	Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer that points to the last character in it. It opens only in the append mode. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file.
"r+"	Searches file. It is opened successfully fopen( ) loads it into memory and sets up a pointer that points to the first character in it. Returns NULL, if unable to open the file.
"w+"	Searches file. If the file exists, its contents are overwritten. If the file doesn't exist a new file is created. Returns NULL, if unable to open the file.
"a+"	Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer that points to the last character in it. It opens the file in both reading and append mode. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file.

Binary File Modes

Mode	Meaning
"rb"	Read binary
"wb"	Write binary
"ab"	Append binary
"rb+" or "r+b"	Read & write binary
"wb+" or "w+b"	Read & write binary (overwrite)
"ab+" or "a+b"	Read & append binary

Mode	File Must Exist	File Created	Existing Data Lost
`r`	Yes	No	No
`w`	No	Yes	Yes
`a`	No	Yes	No
`r+`	Yes	No	No
`w+`	No	Yes	Yes
`a+`	No	Yes	No

Open a File

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE* fptr; // File pointer to store the value returned by fopen
    fptr = fopen(“sam.txt", "r"); // Opening the file in read mode
    if (fptr == NULL)   // checking if the file is opened successfully
        printf("The file is not opened.");
     else
        printf("The file is created Successfully.");
    return 0;
}
```

Create a File

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE* fptr; // File pointer
   fptr = fopen("file.txt", "w"); // Creating file using fopen() with access mode "w"
   if (fptr == NULL) // checking if the file is created
      printf("The file is not opened.");
   else
      printf("The file is created Successfully.");
   return 0;
}
```

Write to a File

Function	Description
fprintf()	Similar to printf(), this function uses formatted string and variable arguments list to print output to the file.
fputs()	Prints the whole line in the file and a newline at the end.
fputc()	Prints a single character into the file.
fputw()	Prints a number to the file. (Not recommended)
fwrite()	This function writes the specified number of bytes to the binary file.

fprintf()

```
#include<stdio.h>
int main()
{
    int i;
    char str[50];
   FILE *fptr = fopen("names.txt", "w"); //open file sample.txt in write mode
    if (fptr == NULL)
    {
        printf("Could not open file");
        return 0;
    }
  printf("Enter names\n");
    for (i = 1; i <= 3; i++)
    {
        scanf("%s", str);
        fprintf(fptr,"%s\n", str);
    }
    fclose(fptr);
    return 0;
}
```

fputs()

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[20];
    scanf("%s",str);
    FILE* fp;
    fp = fopen(“sample1.txt", "w");
    fputs(str, fp);
    fclose(fp);
    return 0;
}
```

fputc()

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[20];
    scanf("%s",str);
    FILE* fp;
    fp = fopen("sample2.txt", "w");
    int i=0;
    do{
        fputc(str[i],fp);
        i++;
    }
    while(str[i]!='\0');
    fclose(fp);
    return 0;
}
```

fputc()

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[20];
    scanf("%s",str);
    FILE* fp;
    fp = fopen("sample2.txt", “a");
    int i=0;
    do{
        fputc(str[i],fp);
        i++;
    }
    while(str[i]!='\0');
    fclose(fp);
    return 0;
}
```

Read from a file

Function	Description
fscanf()	Reads formatted input (int, float, string, etc.) from a file similar to scanf()
fgets()	Reads up to n-1 characters or until a newline (\n) is found
fgetc()	Reads one character from the file and moves the file pointer forward. Returns the character or EOF on end-of-file/error.
fgetw()	Prints a number to the file. (Not recommended)
fread()	Reads blocks of binary data (structures, arrays) from a file.

fscanf()

```
#include <stdio.h>
int main() {
    FILE *fp;
    int id;        float marks;        char name[20];
    fp = fopen("sample.txt", "r");
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }
    fscanf(fp, "%d %f %s", &id, &marks, name);
    printf("ID: %d\n", id);
    printf("Marks: %.2f\n", marks);
    printf("Name: %s\n", name);
    fclose(fp);
    return 0;
}
```

```c
#include <stdio.h>
int main() {
    FILE *fp;
    int id;        float marks;        char name[20];
    fp = fopen("sample2.txt", "r");
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }
   while (!feof(fp)) {
    if (fscanf(fp, "%d %f %19s", &id, &marks, name) == 3) {
        printf("ID: %d\n", id);
        printf("Marks: %.2f\n", marks);
        printf("Name: %s\n\n", name);
    }
}
    fclose(fp);
    return 0;
}
```

fgets()

```
#include <stdio.h>
int main() {
    FILE *fp;
    char line[100];
    fp = fopen("sample.txt", "r");
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }
    while (fgets(line, sizeof(line), fp) != NULL) {
        printf("%s", line);
    }
    fclose(fp);
    return 0;
}
```

It stores the input into a character array and stops reading when it reaches a newline character, the specified number of characters, or end-of-file (EOF).

Returns NULL if an error occurs or the end-of-file (EOF) is reached.

fgetc()

```
#include <stdio.h>
int main() {
   FILE *fp;
   char ch;
   fp = fopen("sample.txt", "r");
   if (fp == NULL) {
      printf("File not found\n");
      return 1;
   }
    while ((ch = fgetc(fp)) != EOF)
      printf("%c", ch);
   fclose(fp);
   return 0;
}
```

fgetc() is used to obtain input from a file single character at a time. This function returns the ASCII code of the character read by the function.

It returns the character present at position indicated by file pointer. After reading the character, the file pointer is advanced to next character.

If pointer is at end of file or if an error occurs EOF file is returned by this function.

```c
#include <stdio.h>
int main() {
    FILE *fp;
    int num, count = 0;
    float sum = 0.0, avg;

    /* Open file in read mode */
    fp = fopen("numbers.txt", "r");
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }

    /* Read numbers and calculate sum */
    while (fscanf(fp, "%d", &num) == 1) {
        sum += num;
        count++;
    }

    fclose(fp);
```

```c
    if (count == 0) {
        printf("No numbers in file\n");
        return 1;
    }

    avg = sum / count;

    /* Reopen same file in append mode */
    fp = fopen("numbers.txt", "a");
    if (fp == NULL) {
        printf("File cannot be opened for writing\n");
        return 1;
    }

    /* Write result to the same file */
    fprintf(fp, "\nAverage = %.2f\n", avg);

    fclose(fp);
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>   // for atoi()

int main() {
    FILE *fp;
    char line[100];
    int num, count = 0;
    float sum = 0.0, avg;

    /* Open file in read mode */
    fp = fopen("numbers.txt", "r");
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }

    /* Read each line using fgets */
    while (fgets(line, sizeof(line), fp) != NULL) {
        num = atoi(line);   // convert string to integer
        sum += num;
        count++;
    }

    fclose(fp);
```

```c
    if (count == 0) {
        printf("No numbers in file\n");
        return 1;
    }

    avg = sum / count;

    /* Open same file in append mode */
    fp = fopen("numbers.txt", "a");
    if (fp == NULL) {
        printf("File cannot be opened for writing\n");
        return 1;
    }

    /* Write average to the same file */
    fprintf(fp, "\nAverage = %.2f\n", avg);

    fclose(fp);
    return 0;
}
```

Seek

- seek means moving the file pointer to a specific position inside a file so that the next read or write happens from that position.

```
int fseek(FILE *fp, long offset, int origin);
```

- fseek() returns 0 on success, non-zero on failure
- Offset is measured in bytes
- Mainly used in binary files
- Enables random access file handling

- Origin
- SEEK_SET → beginning of file
- SEEK_CUR → current position
- SEEK_END → end of file

```c
#include <stdio.h>

int main() {
    FILE *fp;
    char ch;

    /* Create file and write data */
    fp = fopen("seekdemo.txt", "w+");
    if (fp == NULL) {
        printf("File cannot be opened\n");
        return 1;
    }

    fputs("ABCDEFGHIJ", fp);   // 10 characters

    /* ---------- SEEK_SET ---------- */
    fseek(fp, 0, SEEK_SET);    // Move to beginning
    ch = fgetc(fp);
    printf("Using SEEK_SET (from beginning): %c\n", ch);
```

```c
 /* ---------- SEEK_CUR ---------- */
    fseek(fp, 4, SEEK_CUR);    // Move 4 bytes forward from
current
    ch = fgetc(fp);
    printf("Using SEEK_CUR (from current): %c\n", ch);

    /* ---------- SEEK_END ---------- */
    fseek(fp, -1, SEEK_END);   // Move to last character
    ch = fgetc(fp);
    printf("Using SEEK_END (from end): %c\n", ch);

    fclose(fp);
    return 0;
}
```

```
ABCDEFGHIJ
```

```
Using SEEK_SET (from beginning): A
Using SEEK_CUR (from current): F
Using SEEK_END (from end): J
```

```c
#include <stdio.h>

int main() {
    FILE *fp;
    char ch;

    /* Create and write to a file */
    fp = fopen("demo.txt", "w+");
    if (fp == NULL) {
        printf("File cannot be opened\n");
        return 1;
    }

    fputs("Welcome to File Handling in C", fp);

    /* Move file pointer 11 bytes from beginning */
    fseek(fp, 11, SEEK_SET);
```

```c
    printf("Data after fseek():\n");

    /* Read characters from the new position */
    while ((ch = fgetc(fp)) != EOF) {
        putchar(ch);
    }

    fclose(fp);
    return 0;
}
```

```
Welcome to File Handling in C
```

```
Data after fseek():
File Handling in C
```

fwrite()

- fwrite() is a binary file input/output function in C used to write data directly to a file in binary form.
- It writes data exactly as it is stored in memory (byte by byte).
- size_t fwrite(const void *ptr, size_t size, size_t count, FILE *fp);

Parameter	Meaning
`ptr`	Address of the data to be written
`size`	Size of one data item (in bytes)
`count`	Number of data items
`fp`	File pointer

Returns **number of items successfully written**
If return value < count → write error occurred

```c
#include <stdio.h>
struct student {
    int id;
    char name[30];
    float marks;
};
int main() {
    FILE *fp;
    struct student s;
     printf("Enter Student ID   : ");          scanf("%d", &s.id);
    printf("Enter Student Name : ");       scanf(" %[^\n]", s.name);
    printf("Enter Marks        : ");       scanf("%f", &s.marks);
    fp = fopen("student.dat", "wb"); /* Open file in write binary mode */
    if (fp == NULL) {
        printf("File cannot be opened\n");
        return 1;
    }
    fwrite(&s, sizeof(struct student), 1, fp); /* Write data to file */
    printf("\nData written to file successfully.\n");
    fclose(fp);
    return 0;
}
```

fread()

- fread() is used to read data from a binary file into program memory in blocks of fixed size.
- size_t fread(const void *ptr, size_t size, size_t count, FILE *fp);

Parameter	Meaning
`ptr`	Address of the data to be written
`size`	Size of one data item (in bytes)
`count`	Number of data items
`fp`	File pointer

Returns the number of blocks successfully read.

```
#include <stdio.h>
struct student {
    int id;
    char name[30];
    float marks;
};
int main() {
    FILE *fp;
    struct student s;
    fp = fopen("student.dat", "rb");   // Read binary mode
    if (fp == NULL) {
        printf("File cannot be opened\n");
        return 1;
    }
    printf("Student Records:\n");
    while (fread(&s, sizeof(struct student), 1, fp) == 1) {/* Read until fread returns 0 */
        printf("ID    : %d\n Name  : %s\n Marks : %.2f\n ", s.id, s.name, s.marks);
    }
```

Command Line Arguments

- Command line arguments are values passed to a C program at the time of execution, through the command prompt or terminal.
- They allow a program to take input without using scanf().
- Syntax int main(int argc, char *argv[])
- argc (Argument Count)
  - Stores the total number of command line arguments.
  - Includes the program name.
- argv (Argument Vector)
  - An array of strings (char *).
  - Each element stores one command line argument.
  - argv[0] → Program name
  - argv[1] → First argument
  - argv[2] → Second argument
  - Arguments are always strings

Example

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    int a, b;
    if (argc != 3) {
        printf("Usage: program num1 num2\n");
        return 1;
    }
    a = atoi(argv[1]);
    b = atoi(argv[2]);
    printf("Output of the program %s is %d\n", argv[0], a + b);
    return 0;
}
```